

CS 4610 HW 3

What is the "same-origin policy"? (10pts)

Same-Origin Policy is a critical web browser security mechanism designed to isolate potentially malicious documents and prevent unauthorized data access across different websites. Under this policy, a web browser will only permit scripts contained in one web page to read or modify data in a second web page if both pages share the exact same origin. An origin is strictly defined by three matching components: the protocol, the host or domain name, and the port number. If any of these three elements differ between two URLs, the browser blocks the interaction.

RJ, a web programmer for a bank, has been given the task of building the online banking web site. He needs to create an authentication web application that accepts user input of username and passwords and verifies the input. RJ is told to use authentication cookies so that the web application does not frequently re-authenticate users. He generates a random number S only once, and then increases S by a constant value of $0x5f$ to create the cookie for each user. For example, the authentication cookies for user 1, 2, 3, ... are $S + 0x5f$, $S + 0x5f + 0x5f$, $S + 0x5f + 0x5f + 0x5f$..., respectively. Please describe an attack that exploits RJ's implementation to compromise other users' accounts and discuss a possible solution to address this attack. (10 pts)

RJ's implementation is heavily flawed and vulnerable to a Session Hijacking attack via predictable tokens. Because the authentication cookies are generated sequentially using a fixed mathematical pattern,

an attacker can easily deduce the active session identifiers of other bank customers.

To execute the attack, the adversary simply logs into their own bank account to receive a valid session cookie. Once they have their assigned token, they can add or subtract $0x5f$ to accurately predict the exact cookie values of users who logged in immediately before or after them. The attacker can then use their browser's developer tools or a command-line script to overwrite their own cookie with the victim's predicted value. When the attacker navigates to the bank's account dashboard, the server reads the forged cookie, assumes the attacker is the victim, and grants them full access to the victim's funds without ever requiring a password.

To properly address and prevent this attack, RJ must abandon the sequential counter method and instead generate highly unpredictable session tokens. The standard solution is to use a cryptographically secure, randomly generated 128-bit string for every new session upon login. Because the token space is so massive, it becomes mathematically unfeasible for an attacker to randomly guess or calculate another user's active session ID. Furthermore, instead of writing a custom generation algorithm, RJ should rely on the robust, built-in session management libraries provided by modern web frameworks, which automatically handle secure random token generation and tie them securely to server-side memory.

Textbook page 198: Consider a scenario in which an attacker is interested in breaking into Windows NT or UNIX shell accounts. Analyze the technical issues surrounding the password security that these operating systems offer. State your assumptions.

How many bits of information are possible in an eight-character password if any character can be used? (Assume each character consists of 8 bits) (3pts)

If any character can be used, and each character consists of 8 bits, each character position has $2^8 = 256$ possible options.

Because the password is 8 characters long, the total number of bits of information is simply the number of characters multiplied by the number of bits per character:

8 characters $\times$ 8 bits/character = 64 bits of information.

How many different combinations of passwords are there in an eight-character password if only the uppercase and lowercase characters plus the ten decimal digits can be used? (3pts)

Uppercase letters: 26

Lowercase letters: 26

Decimal digits: 10

Total character pool = 62 characters

Since the password is exactly 8 characters long, we calculate the number of combinations by raising the character pool to the power of the password length:

$62^8 = 218340105584896$ combinations

How many different combinations of passwords are there if users choose their passwords to be concatenations of dictionary words from a dictionary with the characteristics in Table 11-1? (4pts)

use a dynamic programming approach where we calculate the number of valid password combinations for every length from 1 up to 8.

Let $C(n)$ be the total number of combinations to create a string of exactly length n .

$N_1 = 5$ (1-character words)

$N_2 = 32$ (2-character words)

$N_4 = 103$ (4-character words)

$N_5 = 402$ (5-character words)

There are exactly 24,955,821 different combinations of passwords that can be created by concatenating these specific dictionary words into an 8-character string.

Table 11-1. Dictionary of Words

Number of Characters in Word	Number of Words
1	5
2	32
4	103
5	402

Would it be possible to thwart all possible password cracking attacks by increasing the salt length to something large, like 64 bits?
(10pts)

No, increasing the salt length to 64 bits would not thwart all possible password cracking attacks. A salt is a random string of data appended to a password before it is hashed. Its primary purpose is to ensure that two users with the exact same password will have completely different hashes, which effectively prevents the use of precomputed rainbow tables. A 64-bit salt makes generating a rainbow table mathematically infeasible due to the sheer size required. However, the salt is traditionally stored in plaintext right next to the hashed password in the database. If an attacker breaches the database and targets a specific user, they already know the salt for that user. They can simply append that known 64-bit salt to every guess in their dictionary and run an offline brute-force or dictionary attack against that specific hash. Furthermore, a long salt provides absolutely no protection against online guessing attacks, keyloggers, phishing, or social engineering.

What is an Advanced Persistent Threat (APT)? Which tool we have discussed so far can be used to mitigate this threat? Please explain this tool. (10pts)

An Advanced Persistent Threat is a sophisticated, prolonged cyberattack in which an intruder gains access to a network and remains undetected for an extended period. Rather than causing immediate, loud damage, APTs are typically executed by highly resourced adversaries with the goal of slowly exfiltrating sensitive data. To stay hidden, APTs frequently utilize stealth malware and kernel-level rootkits to alter the victim's operating system, making their malicious processes and network connections invisible to traditional, in-OS antivirus software. The tool discussed so far is Virtual Machine Introspection, which is utilized by a Virtual Machine Monitor. Because the VMM sits underneath the Guest OS and controls the physical hardware, it can inspect the Guest OS's raw memory from the outside without relying on the compromised operating system's internal APIs. By counting the actual running processes in the raw memory and comparing it to the list of processes the Guest OS reports. If there is a mismatch, the VMM knows an APT rootkit is hiding stealth malware and can immediately terminate the compromised virtual machine.

Programmer RJ tries to design a network protocol to enable a client to check whether a server is still active. He creates a protocol in this way. The client generates a random string and sends the string together with the string length to the server. If the server is still active, the server will echo the string back to the client so that the client knows that the connection is not lost. The server uses the `string_process` function below to process the string. The function first creates a buffer of the size equal to `strlen`, which is the string length sent by the client, and then it copies the input string (the one sent by the client) of size `strlen` to the buffer. The `echo_string` function sends the content in the buffer to the client. This code has a vulnerability that can be exploited by an adversary to cause severe consequences. Please identify the vulnerability, discuss a solution, and explain your reason. (10pts)

```
void string_process (unsigned char *string, unsigned int strlen)

{
    unsigned char *buffer;
    buffer = (unsigned char *) malloc (strlen);
    memcpy (buffer, string, strlen);
    echo_string (buffer, strlen);
    Return;
}
```

The vulnerability in RJ's code is an Out-of-Bounds Read / Information Leak. The core issue is that RJ blindly trusts the client-provided strlen variable without verifying if it matches the actual length of the string sent by the client. Because malloc and memcpy strictly use the client's strlen value, an attacker can exploit this by sending a very short string but declaring a massive strlen (e.g., 64,000). The server will allocate 64,000 bytes and blindly copy 64,000 bytes starting from the memory address of the 1-byte string. This forces the server to read way past the boundaries of the intended buffer and scoop up 63,999 bytes of adjacent server memory—which could contain highly sensitive data like passwords, session cookies, or private encryption keys. The echo_string function then packages up all this stolen memory and sends it right back to the attacker. To solve this, the server must never trust client-provided length variables. The solution is to have the server independently calculate the length of the received string payload upon arrival and enforce a strict, hard-coded maximum length limit before allocating memory or copying data.

If an organization uses network-based IDS to detect attacks, and it can monitor all incoming and outgoing traffic, is there any value to also using host-based IDS? If so, why, and if not, why not? (10pts)

Yes, there is immense value in using a Host-based Intrusion Detection System alongside a Network-based IDS, as it satisfies the core

security principle of defense in depth. While a NIDS is excellent at spotting malicious packets traversing the wire, it suffers from a major blind spot: encrypted traffic. If an attacker sends a malicious payload over an HTTPS/TLS connection, the NIDS cannot read the encrypted contents. A HIDS, however, monitors activity directly on the endpoint machine where the data is ultimately decrypted and processed. Furthermore, a NIDS cannot detect local threats that never cross the network, such as a rogue employee inserting a malicious USB drive or a user escalating their local privileges. A HIDS monitors deep system context that a NIDS cannot see, such as unauthorized modifications to system files, unexpected registry changes, or a program making suspicious system calls, allowing it to catch an intrusion even if the initial network attack went undetected.

One approach to defeat the tiny fragment attack against firewall packet filtering is to enforce a minimum length of the transport header that must be contained in the first fragment of an IP packet. If the first fragment is rejected, all subsequent fragments can be rejected. However, the nature of TCP is that fragments may arrive out of order. Thus, an intermediate fragment may pass through the filter before the initial fragment is rejected. How can this situation be handled? (10pts)

If an intermediate or subsequent fragment arrives at a stateless firewall before the initial fragment, the firewall might indeed allow it to pass through to the internal network, as it has no context that the associated first fragment is malicious. However, this situation naturally resolves itself at the destination host and does not result in a successful attack. Because the firewall strictly enforces the minimum header length rule, the tiny first fragment will inevitably be dropped when it finally arrives. Without this first fragment containing the start of the TCP header, the receiving host's operating system cannot complete the IP fragmentation reassembly process. The intermediate fragments that slipped through will simply sit in the destination host's memory buffer until the IP reassembly timer expires, at which point the host will silently discard them.

Alternatively, modern stateful firewalls handle this elegantly by performing virtual reassembly directly on the firewall itself. In this scenario, the firewall holds all out-of-order fragments in its own memory and refuses to forward any of them to the internal network until the entire packet has been reassembled, inspected, and verified to contain a valid, full-length transport header.

Eve, a hacker, uses some automatic tool to scan existing popular web pages to search for the SQL injection vulnerability. Eve finds a webpage with the URL below

<https://oceanbaynews.com/music.php?id=3153>

From the URL, Eve guesses that this URL sends a search query to some database to search a table of the name 'music'. The search query may look like this

```
Select * from music where id = 3153
```

Eve wants to know what information is stored in this database. So, he wants to know the names of all tables, and the names of all table columns. Please compose malicious URL with the SQL injection payload that can display this information to Eve. Please also propose a defense method to prevent this attack. (10pts)

To extract the database schema, Eve would utilize a UNION-based SQL injection attack targeting the information_schema.columns table, which is a standard metadata repository present in most modern relational databases.

Because a UNION operator requires both queries to have the exact same number of columns, Eve would first systematically test the URL to find the column count, and then inject her payload. Assuming the original music table has two columns being displayed, Eve could use the following malicious URL:

`https://oceanbaynews.com/music.php?id=-1%20UNION%20SELECT%20table_name,%20column_name%20FROM%20information_schema.columns%20--` (where %20 represents URL-encoded spaces).

By changing the initial ID to a non-existent record like -1, Eve forces the original query to return empty, ensuring that the web page only displays the results of her injected query, which will print out every table name and column name in the database. To completely prevent this attack, the web developer must use Parameterized Queries. By binding the id parameter, the database engine compiles the SQL structure first and treats the attacker's input strictly as a literal data value. This ensures that the injected SQL commands are never interpreted as executable code, entirely neutralizing the vulnerability.